

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

Control Structures

L20 - Selection Statements- Translation between if and switch

Department of Computer Science
University of Pretoria

- You can use either if-statements or switch-statements to perform actions conditionally.
- Which one you use is dependent on the application it is being used for.
- There are instances when one selection statement is more appropriate in the given situation than another and a translation between the two is required.

- We will consider the translation from an if-statement to a switch-statement followed by the reverse.
- If-statements are more versatile and can evaluate a wide range of conditions, based on a range of variables.
- However, when you have a single variable being tested against multiple values, a switch-statement often

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

becomes more readable and
straightforward.

Consider the following if-else chain that checks an integer variable:

Listing 1: If-Else Chain To Display Status

```
int status = getStatus(); // Assume getStatus() returns an int

if (status == 1) {
    cout << "Pending";
} else if (status == 2) {
    cout << "Approved";
} else if (status == 3) {
    cout << "Rejected";
} else {
    cout << "Unknown-status";
}
```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

This can be directly translated into a switch-statement as follows:

Listing 2: Switch-Statement Equivalent To Display Status

```
int status = getStatus(); // Assume getStatus() returns an int
switch (status) {
    case 1:
        cout << " Pending";
        break;
    case 2:
        cout << " Approved";
        break;
    case 3:
        cout << " Rejected";
        break;
    default:
        cout << " Unknown-status";
}
```

- The switch-statement is cleaner and easier to manage, particularly as the number of conditions grows.
- Note that the translation from if-statement to switch-statement is only possible when all conditions compare against a single variable/expression.
- While switch-statements are useful for checking a single variable against a

series of constants, they are limited to equality checks.

- To handle more complex conditions (eg. ranges) or to involve logical operators, translating to if-statements may be necessary.

Consider a switch-statement that handles different grades given in Listing 3.

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

Listing 3: Switch-Statement for Grades

```
char grade = getGrade(); // Assume getGrade() returns a char

switch (grade) {
    case 'A':
        cout << "Excellent";
        break;
    case 'B':
    case 'C':
    case 'D':
        cout << "You-passed";
        break;
    case 'F':
        cout << "Better-luck-next-time";
        break;
    default:
        cout << "Invalid-grade";
}
```

- The translation of the given switch-statement to an if-statements needs to take the ranges of distinct values into account.
-
- A grade of B, C and D all result in an output of “You passed”.
-

- This effective is combination of testing of the logical disjunction (the || operator) of the equivalences of grade with each of the grade symbols, B, C and D.

Note the use of the chaining to create the equivalent statement.

Listing 4: If-Statement Equivalent for Grades

```
if (grade == 'A') {
```

```
    cout << "Excellent";  
} else if (grade == 'B' || grade == 'C' || grade == 'D') {  
    cout << "You-passed";  
} else if (grade == 'F') {  
    cout << "Better-luck-next-time";  
} else {  
    cout << "Invalid-grade";  
}
```

- An if-statement can handle a broader range of conditions, such as checking for a range of values or combining multiple conditions, which is not possible with a switch-statement.

- If you need to check ranges or conditions that involve more than direct equality, you generally use if-statements.
- Considerations that need to be taken into account when translating between if-statements and switch-statement.
- 1 ● **Use ‘switch’ when:** You have a single variable being compared against multiple distinct constants.

- 2 **Use 'if' when:** Conditions are complex, involve logical operators, or you are checking ranges.
- Translate Listing ?? to the equivalent chained else-if format.
- Revisit Nested if-statements, Remember they occur when an if or else if block contains another if-statement. This nesting can continue to any level of

Translating Between If and switch

Nested If to switch

Ternary If to switch

Ternary Operator to switch with Functions

Nested Ternary Operator to switch

Program Segment to switch

depth as needed.

Translating Between If and switch

Nested If to switch

Ternary If to switch

Ternary Operator to switch with Functions

Nested Ternary Operator to switch

Program Segment to switch

Listing 5: True Nested If-Statements

```
#include <iostream>
using namespace std;

int main() {
    int age;
    cout << "Type in an age [0 to 120]: ";
    cin >> age;

    if (age < 35) {
        cout << "You are young" << endl;
    } else {
        if (age < 65) { // age >= 35 and age < 65
            cout << "You are middle-aged" << endl;
        } else {
            cout << "You are ancient" << endl;
        }
    }

    return 0;}
```

- The 0 ... N construct in C++ is called a "range-based case".
- It allows you to specify a range of values directly in a case statement, making the code more concise and readable.

Listing 6: True Nested If-Statements to Case statement

```
#include <iostream>  
using namespace std;
```

Translating Between If and switch

Nested If to switch

Ternary If to switch

Ternary Operator to switch with Functions

Nested Ternary Operator to switch

Program Segment to switch

```
int main() {  
    int age;  
    cout << "Type in an age [0 to 120]: ";  
    cin >> age;  
  
    switch (age) {  
        case 0 ... 34:  
            cout << "You are young" << endl;  
            break;  
        case 35 ... 64:  
            cout << "You are middle-aged" << endl;  
            break;  
        default:  
            cout << "You are ancient" << endl;  
            break;  
    }  
    return 0;}
```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

Listing 7: Using the Ternary Operator in C++

```
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two numbers:\n";
    cin >> a >> b;
    int max = (a > b) ? a : b;
    cout << "The greater number is -" << max << endl;
    return 0;
}
```

Listing 8: switching Ternary Operator to switch in C++

```
#include <iostream>
```

Translating Between If and switch

Nested If to switch

Ternary If to switch

Ternary Operator to switch with Functions

Nested Ternary Operator to switch

Program Segment to switch

```
using namespace std;
```

```
int main() {  
    int a, b;  
    cout << "Enter two numbers:\n";  
    cin >> a >> b;  
  
    switch(a > b) {  
        case true:  
            cout << "The greater number is-" << a << endl;  
            break;  
        case false:  
            cout << "The greater number is-" << b << endl;  
            break;  
    }  
  
    return 0;  
}
```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

Listing 9: Ternary Operator conversion with Function Calls

```
#include <iostream>
using namespace std;

int add(int a, int b) {
    return a + b;
}

int subtract(int a, int b) {
    return a - b;
}

int main() {
    int num1, num2;
    char operation;
    cout << "Enter two numbers: ";
    cin >> num1 >> num2;
    cout << "Choose operation - (a for add, s for subtract): ";
```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

```
cin >> operation;  
  
bool condition = (operation == 'a');  
  
int result = condition ?  
    add(num1, num2) : subtract(num1, num2);  
  
cout << "The result is:-" << result << endl;  
  
return 0;  
}
```

Listing 10: Ternary Operator conversion to switch with Function Calls

```
#include <iostream>  
using namespace std;
```

Translating Between If and switch

Nested If to switch

Ternary If to switch

Ternary Operator to switch with Functions

Nested Ternary Operator to switch

Program Segment to switch

```
int add(int a, int b) {  
    return a + b;  
}  
  
int subtract(int a, int b) {  
    return a - b;  
}  
  
int main() {  
    int num1, num2;  
    char operation;  
    cout << "Enter two numbers: -";  
    cin >> num1 >> num2;  
    cout << "Choose operation - (a for add, -s for subtract) : -";  
    cin >> operation;  
  
    int result;  
    switch (operation) {  
        case 'a':  
            result = add(num1, num2);  

```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

```
        break;
    case 's':
        result = subtract(num1, num2);
        break;
    default:
        cout << "Invalid-operation" << endl;
        return 1; // Exit with error code
        //give a reson why break is not applicable ,
//run the code with return 1 and with break to spot the difference
        //break;

}

cout << "The-result-is:-" << result << endl;

return 0;
}
```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

Listing 11: Nested Ternary Operator

```
#include <iostream>
using namespace std;

int main() {
    int score;
    cout << "Enter your score: -";
    cin >> score;

    //Simple nesting
    string grade = (score < 50 ) ? "Fail" :
                   (score < 75) ? "Pass" : "Distinction";

    cout << "Your grade is: -" << grade << endl;

    //Complex nesting
    string detailed = (score >= 90)? "High-Distinction" :
                      (score >= 75) ? "Distinction" :
                      (score >= 60) ? "Credit" :
                      (score >= 50) ? "Pass" : "Fail";
```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

```
    cout << "Your-detailed-grade-is:-" << detailed << endl;  
  
    return 0;  
}
```

Listing 12: Nested Ternary Operator to switch

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int score;  
    cout << "Enter-your-score:-";  
    cin >> score;  
  
    // Simple nesting  
    string grade;  
    switch (score) {
```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

```
case 0 ... 49:
    grade = "Fail";
    break;
case 50 ... 74:
    grade = "Pass";
    break;
default:
    grade = "Distinction";
    break;
}
cout << "Your-grade-is:-" << grade << endl;

// Complex nesting
string detailed;
switch (score) {
    case 90 ... 100:
        detailed = "High-Distinction";
        break;
    case 75 ... 89:
        detailed = "Distinction";
```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

```
        break;
    case 60 ... 74:
        detailed = "Credit";
        break;
    case 50 ... 59:
        detailed = "Pass";
        break;
    default:
        detailed = "Fail";
        break;
}
cout << "Your-detailed-grade-is:-" << detailed << endl;

return 0;
}
```

Listing 13: Program Segment Equivalence

```
#include <iostream>
```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

```
using namespace std;

int firstFunction(int x){
    if (x > 100) {
        cout << x << endl;
        x += 2;
    } else {
        x += 2;
    }

    return x;
}

int secondFunction(int x){
    if (x > 100) {
        cout << x << endl;
    }
    x += 2;

    return x;
}
```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

```
}  
  
int main() {  
    int x;  
    cout << "Enter a value:-";  
    cin >> x;  
    firstFunction(x);  
    // versus  
    secondFunction(x);  
  
    return 0;  
}
```

Listing 14: Program Segment Equivalence translation to switch

```
#include <iostream>  
using namespace std;
```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

```
int firstFunction(int x){  
    switch (x > 100) {  
        case true:  
            cout << x << endl;  
            x += 2;  
            break;  
        case false:  
            x += 2;  
            break;  
    }  
    return x;  
}  
  
int secondFunction(int x){  
    switch (x > 100) {  
        case true:  
            cout << x << endl;  
            break;  
    }  
}
```

Translating Between If and switch
Nested If to switch
Ternary If to switch
Ternary Operator to switch with Functions
Nested Ternary Operator to switch
Program Segment to switch

```
    x += 2;
    return x;
}

int main() {
    int x;
    cout << "Enter a value: -";
    cin >> x;
    firstFunction(x);
    // versus
    secondFunction(x);

    return 0;
}
```